



**University of Manchester
School of Computer Science**

Drawing Venn Diagrams

Author: Murad Rzazade

Supervisor: Dr. Ian Pratt-Hartmann

Degree: BSc(Hons) Computer Science with Business and Management

**Project Report
May, 2016**

Abstract

Venn diagrams are a type of diagrams used to visually represent formulas of propositional logic, or, alternatively, Boolean relations over some collection of sets. The aim of this project is to build an application that will draw the Venn diagram for any number of logical variables, or, alternatively for any number of sets. This problem involves parsing propositional formulas, computing their truth tables and displaying the associated Venn diagrams.

Project Title: Drawing Venn Diagrams

Author: Murad Rzazade

Degree: Computer Science with Business and Management

Supervisor: Dr. Ian Pratt-Hartmann

Keywords: Venn diagrams, Diagrammatic representation, Truth table, Logical relations.

Acknowledgement

I would like to thank my supervisor Dr. Ian Pratt-Hartmann for his continuous support and guidance he has given me over the work on this project.

I would also like to thank my family for their belief in me.

Contents

1	Introduction	5
1.1	Aim and Objective.....	5
1.2	Overview of the Program.....	6
1.3	Overview of the Report.....	6
2	Context and Background	7
2.1	Boolean Algebra	7
2.1.1	Basic Operations of Boolean Algebra.....	7
2.1.2	Derived Operations of Boolean Algebra.....	8
2.2	Venn Diagrams	8
2.2.1	Symmetric Diagrams.....	10
2.2.2	Edward's Construction.....	10
2.2.3	Venn' Construction	11
2.2.4	Relation between the Truth table and the Venn diagram.....	12
3	Design and Implementation	13
3.1	Program features	13
3.2	Design.....	15
3.2.1	Truth table package	15
3.2.2	Venn diagram package	16
3.3	Implementation.....	17
3.3.1	Generating the Truth table	18
3.3.2	Drawing n-curved Venn diagram.....	20
3.3.3	Mapping.....	24
3.3.4	Filling the regions.....	28
3.3.5	Zooming	30
4	Testing and Evaluation	32
4.1	Testing.....	32
4.2	Evaluation	33
5	Conclusion and Future Works	36
5.1	Conclusion	36
5.2	Future Works	36

List of Figures

1.1	Final output of the program.....	6
2.1	Truth table for basic Boolean algebra	7
2.2	Truth table for derived Boolean algebra	8
2.3	Venn diagrams for 2 and 3 sets respectively	9
2.4	Diagram using 4 circles	9
2.5	Symmetric Venn diagrams for $n = 5, 7$ and 11	10
2.6	Edward's construction for $n = 1, 2, 3...6$	11
2.7	Venn's construction for $n = 1, 2, 3...6$	11
2.8	Relation between the truth table and the Venn diagram	12
3.1	End result of the project.....	13
3.2	“Missing connective” error	14
3.3	JComboBox of the input history	14
3.4	Quick guide of usage	15
3.5	Precedence of logical operations	16
3.6	System class diagram of the program	17
3.7	Precedence of logical operators implemented in the project	18
3.8	Pseudo code of Shunting Yard algorithm	18
3.9	Truth table for $A \& B \mid C$	19
3.10	Drawing 3 squares	21
3.11	Adding the first curve from initial 4 points a.....	21
3.12	Adding the first curve from initial 4 points	22
3.13	Computing new points from old points.....	23
3.14	Code for drawing starting and ending lines.....	24
3.15	Code for drawing two lines between the points.....	24
3.16	Relation of the Venn diagram regions with the truth table for $n \leq 3$...	25
3.17	Sectors of the Venn diagram	26
3.18	Venn diagram regions associated with the truth table rows	26
3.19	Order of the last stored points.....	27
3.20	Pair points derived from the path.....	27
3.21	The logic of the Flood fill algorithm.....	29
3.22	Pseudo-code of the Flood fill algorithm.....	29
3.23	Venn diagram before zooming	31
3.24	Venn diagram after zooming	31
4.1	Tabulated runtime results for generating the truth tables	33
4.2	Graphical runtime results for generating the truth tables	33
4.3	Tabulated runtime results for drawing the Venn diagrams	34
4.4	Graphical runtime results for drawing the Venn diagrams	34
4.5	Tabulated runtime results of the whole program	34
4.6	Graphical runtime results of the whole program	35
5.1	Labeling	36

Chapter 1

Introduction

It is intended that this report is read in color.

Diagrams are an essential artifact for reasoning – the primary objective of the diagrams is to clearly represent problems and situations ^[1]. It is always easier to perceive something by use of diagrams rather than in mathematical or linguistic means. Venn diagrams are a type of diagrams used in computer science to visually represent all possible logical relations between different collections of sets.

1.1 Aims and Objectives

The main objective of this project is to write a program where a user inputs a logical formula and gets the diagram that represents it. It would be relatively easy if we knew in advance the maximum variables involved. However, the hard part of this project is to draw the Venn diagrams for any number of sets. Achieving this goal required solving five main technical problems which we refer to as *constructing the truth table*, *drawing*, *mapping*, *filling* and *zooming*.

- I. Creating a truth table involved problems such as parsing the input, and computing all possible truth values for corresponding propositional expression. This part of the project acts as a back end of the application that decides which region will be filled
- II. For drawing part of the project, it was required to write an algorithm that will draw a Venn diagram for any number of sets.
- III. Mapping is a part of the project which links each line of the truth table to its corresponding region in the Venn diagram. This part involved creating an entirely new approach that links all lines of truth table to the regions of the diagram independently of the size of the truth table and the Venn diagram.
- IV. Some regions of the diagrams have to be shaded, to represent relations between the different sets. Shading the regions means to fill a segment or the part of the diagram that is of one color within the borders of another color. This part of the project involved implementing a Flood Fill algorithm.
- V. Zooming is an extra feature of the program. The main reason for implementing zooming was to enable the view of the regions that were impossible to see when the number of sets was big, and there were not enough pixels to show them. This involved implementing special zooming approach that did not involve commonly used scaling for zooming.

1.2 Overview of the program

Figure 1.1 shows the final output of the project. Here user enters a logical formula and gets corresponding Venn diagram and a truth table.

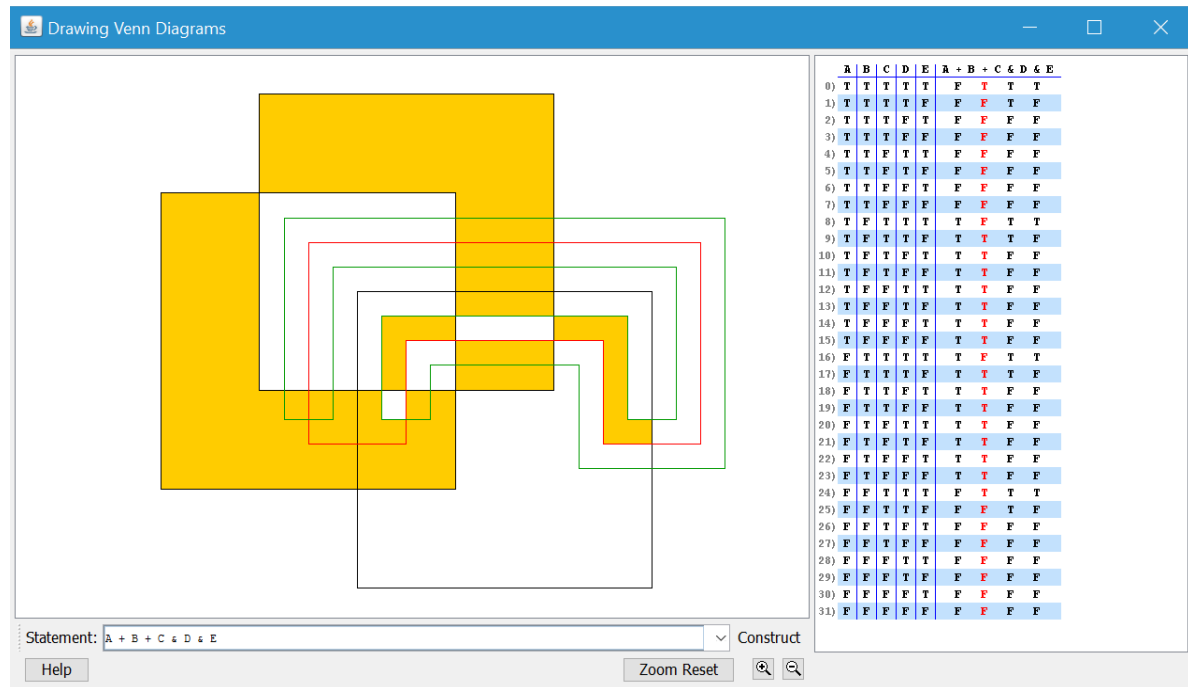


Figure 1.1: Final output of the program.

1.3 Overview of the Report

- **Chapter 2 – Context and Background**

Covers the theories about the Venn Diagrams. Explains existing constructions of the Venn diagrams such as Edward's construction, Venn's construction and symmetric diagrams.

- **Chapter 3 – Design and Implementation**

Explains step by step implementation and approaches that were used in the project to achieve the maximum goal.

- **Chapter 4 – Testing and Evaluation**

Includes test cases to check the validity of the project and performance evaluation of the application.

- **Chapter 5 – Conclusion and Future Works**

Identifies achievements of the project and discusses the possible works for improvement.

Chapter 2

Context and Background

This chapter provides some background information about the Venn diagrams. It explains what the Boolean algebra is and what types of Venn diagrams exist. This chapter also explains the reason for choosing Venn's constructions and its connection to the truth tables.

2.1 Boolean Algebra

Boolean algebra is the part of algebra where variables have truth values. These values are true and false and are denoted by 1 for true and 0 for false. The main operations in Boolean algebra are the negation, disjunction and conjunction, denoted as " $\neg$ ", " $\wedge$ " and " $\vee$ " respectively. Boolean algebra allows us to write logical expressions just as elementary algebra allows to write numeric relations.

2.1.1 Basic Operations of Boolean Algebra

The fundamental operations used in Boolean algebra are AND, OR, NOT:

- AND also called conjunction, denoted as " $\wedge$ ". $X \wedge Y = \text{true (1)}$ only and only if $X = \text{true (1)}$ and $Y = \text{true (1)}$.
- OR also called disjunction, denoted as " $\vee$ ". $X \vee Y = \text{false (0)}$ only and only if $X = \text{false (0)}$ and $Y = \text{false (0)}$.
- NOT also called negation, denoted as " $\neg$ ". $\neg X = \text{true (1)}$ when $X = \text{false (0)}$ and $\neg X = \text{false (0)}$ when $X = \text{true (1)}$.

It is also possible to represent logical relations of $x \wedge y$, $x \vee y$ and $\neg x$ described above in a truth table by writing their values.

x	y	$x \wedge y$	$x \vee y$	x	$\neg x$
0	0	0	0	0	1
1	0	0	1	1	0
0	1	0	1		
1	1	1	1		

Figure 2.1: Truth table for basic Boolean algebra.

2.1.2 Derived Operations of Boolean Algebra

There are three basic Boolean operations described above. And there are three commonly used derived operations that are obtained by combining basic operations. The following examples show how these operations are derived from the basic operations:

$$\begin{aligned}x \rightarrow y &= \neg x \vee y \\x \oplus y &= (x \vee y) \wedge \neg(x \wedge y) \\x \leftrightarrow y &= \neg(x \oplus y)\end{aligned}$$

The first operation denoted as “ $\rightarrow$ ”, is called implication. The second operation denoted as “ $\oplus$ ” is called Exclusive Or (XOR) and the third operation, denoted as “ $\leftrightarrow$ ” is called bi-implication. This symbol sometimes called as logical equality. We can see logical relations of $x \rightarrow y$, $x \oplus y$ and $x \leftrightarrow y$ described above in the truth table by writing their values.

x	y	$x \rightarrow y$	$x \oplus y$	$x \leftrightarrow y$
0	0	1	0	1
1	0	0	1	0
0	1	1	1	0
1	1	1	0	1

Figure 2.2: Truth table for derived Boolean algebra.

Given n operands, the truth table will have 2^n values. In this table, we can see values in rows for each relation. As each relation in this table has 2 operands, each column has 2^2 rows that represent all logical values of the relations.

2.2 Venn Diagrams

Venn diagrams are a diagrammatic representation of logical relations. These diagrams are represented by depicting different sets and showing their relationships. John Venn was the first person who conceived these diagrams. Grünbaum’s theory of independent family can be used to describe what are the Venn diagrams in theory. “Let $S = \{S_1, S_2, S_3 \dots S_n\}$ be a collection of simple closed curves drawn in the plane. The collection S is said to be an independent family if the region formed by the intersection of $X_1, X_2, X_3, \dots, X_n$ is nonempty, where each X_i is either the interior of S_i or is the exterior of S_i . If each such region is connected and there are only finitely many points of intersection between curves, then S is a Venn diagram” [2].

Venn diagrams on two and three sets are shown in figure 2.3. The diagram with the two circles represents the relation between two sets, and the diagram with three circles represents the relation between three sets. The diagram shown on the left has four regions:

A , B , $A \cap B$ (A intersects B), and empty set which is denoted as “ $\emptyset$ ” (the empty set, represents an exterior region of the circles). The diagram shown on the right has 8 regions: 7 are shown in the diagram, and the 8th region is an empty set “ $\emptyset$ ”.

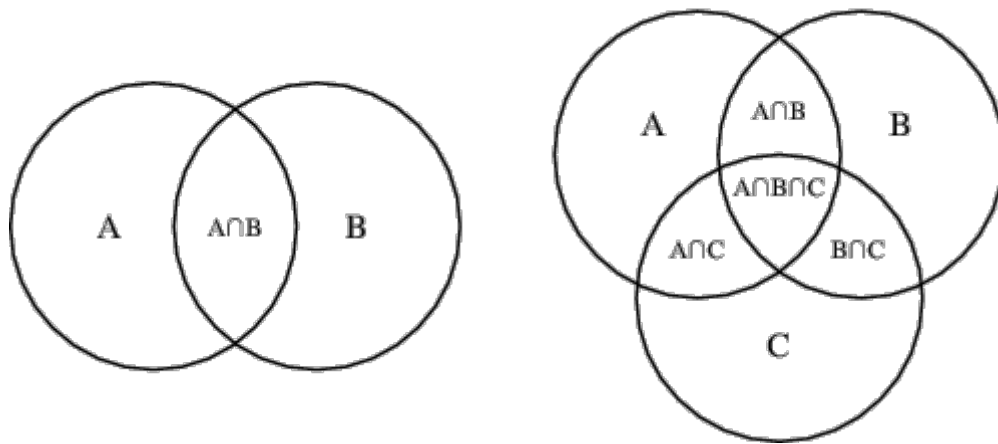


Figure 2.3: Venn diagrams for 2 and 3 sets respectively.

As the Venn diagram is a collection of closed curves such that each curve divides the region into two parts, the number of regions in the Venn diagram equals to 2^n .

From figure 2.3, we can see that it is possible to depict Venn Diagrams using circles for 2 and 3 sets. This brings us to the question of whether it is possible to draw Venn Diagrams for higher n using circles. Let us, consider figure 2.4 that uses 4 circles.

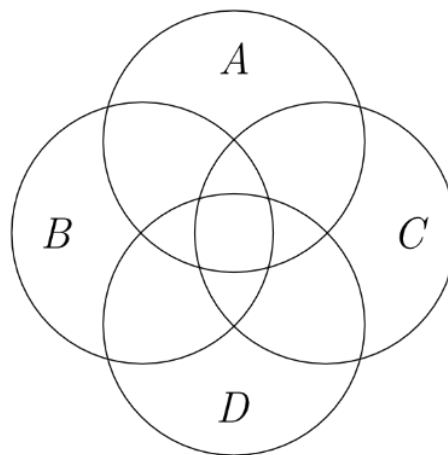


Figure 2.4: Diagram using 4 circles.

If we look carefully at figure 2.4, we can see that it has 13 interior regions and 1 exterior region which make 14 regions in total. However, for Venn diagram exist, there have to be 2^n regions. For this case, there have to be 2^4 regions. So this means that it is not possible to draw Venn diagrams for large n using circles. In order to draw Venn diagrams for big numbers, different constructions are used.

2.2.1 Symmetric Diagrams

The theory of the symmetric diagrams was firstly introduced by Henderson [3]. These diagrams are made from congruent curves and remain unchanged when they are rotated by $2\pi/n$ degrees [3]. They are called symmetric because they remain unchanged after they are rotated. Henderson observed that symmetric diagrams exist only when n is prime.

Many popular symmetric diagrams are used to show logical relations. Most popular of them are diagrams for $n=5$, $n=7$ and recently found a symmetric diagram for $n=11$. Even though these diagrams exist for large n , they are still limited. They exist only for the prime numbers 1,2,3,5,7 and 11. Due to its limitations, it was considered not to use these diagrams in this project.

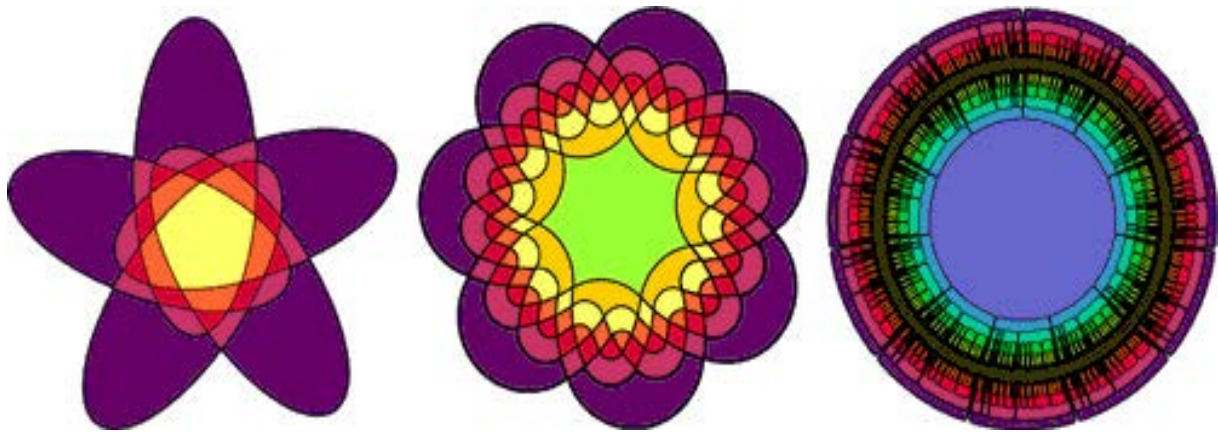


Figure 2.5: Symmetric Venn diagrams for $n = 5, 7$ and 11 [6].

2.2.2 Edward's Construction

Anthony Edward has constructed another diagram to represent logical relations for a higher number of sets. Edward's construction is applicable when representing relations from 1 to 6 sets. It is better to explain his construction by considering figure 2.6. The diagram starts with one line for $n = 1$ and adds a second perpendicular line to it when $n = 2$. For 3 sets a circle is added to the middle of the diagram, where two lines intersect. After, for each new set a new curve is added that divides every existing region into two parts.

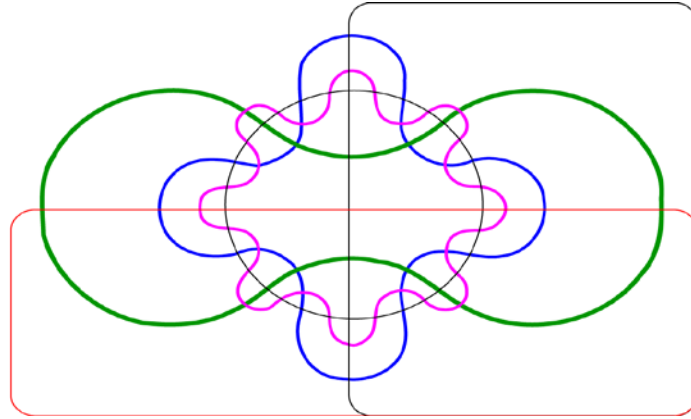


Figure 2.6: Edward's construction for $n = 1, 2, 3...6$ ^[7].

From this figure 2.6, we can see the first line in black color that represents $n = 1$ and second perpendicular line to it in red color which represent $n = 2$. For $n = 3$ a circle is added. After $n = 3$, new curves are added that divide the regions. In this figure curves are added in the order of green, blue and purple that represent $n = 4, 5$ and 6 respectively.

2.2.3 Venn's Construction

As was discussed earlier, several constructions enable drawing Venn diagrams for large n . However, they do not answer the primary question: “Do the Venn diagrams always exist?”, “Can we draw an infinite number of diagrams for any number of n ?”. John Venn proved that these diagrams are always exist by introducing his own construction. His construction starts with elementary 3 circle diagram to represent the relation of 3 sets. Then, new curves are added in a systematic way. Each new curve follows the previous curve and divides all regions into two parts. Figure 2.7 shows Venn's construction. The curves are added in the order of red, green and blue, representing the relation of 4, 5 and 6 sets respectively. The project implements Venn's construction in order to achieve the maximum number of diagrams.

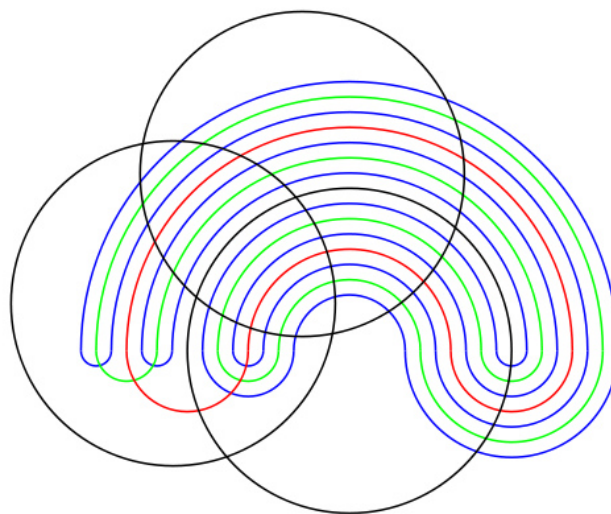


Figure 2.7: Venn's construction for $n = 1, 2, 3...6$ ^[8].

2.2.4 Relation between the Truth table and the Venn diagram

A truth table is an algebraic table used in Boolean algebra to calculate every combination of truth values of a logical function ^[4]. It can be used to find if a logical formula is true or false for every combination of values. A Venn diagram also represents logical functions and their relations by shading some regions of the diagram. A truth table has 2^n rows that show all possible combination of logical function as well as the Venn diagram has 2^n regions that represent all possible relations. The number of truth table rows and the number of Venn diagram regions are equal, and every row of truth table corresponds to one unique region in the Venn diagram. So the truth tables and the Venn diagrams are linked with each other. This project will use the truth table to compute all possible combination of Boolean formula that will enable the program to identify which region(s) of the Venn diagram has to be shaded.

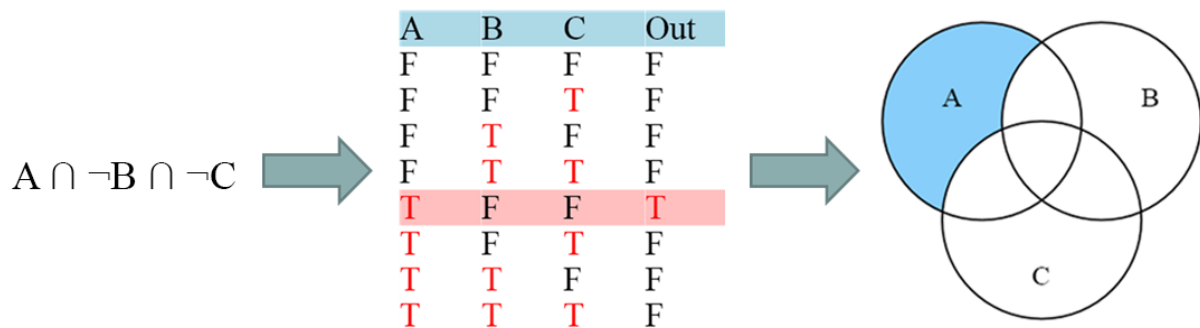


Figure 2.8: Relation between the truth table and the Venn diagram.

Figure 2.8 demonstrates an example of how truth tables and Venn diagrams are linked. In this example, a Boolean formula, $A \cap \neg B \cap \neg C$ (A and not B and not C) is entered. Then a truth table for that formula is generated. As every row of truth table corresponds to one region of Venn diagram, truth table found a highlighted row of where a truth value is true. And it shaded the corresponding region of that row in the Venn diagram.

Chapter 3

Design and Implementation

This chapter shows the features of the program, and then it explains how the program was designed and implemented.

3.1 Program features

In order to better understand how the project works, the report shows screenshots of the application in action. Figure 3.1 shows the end result of the project.

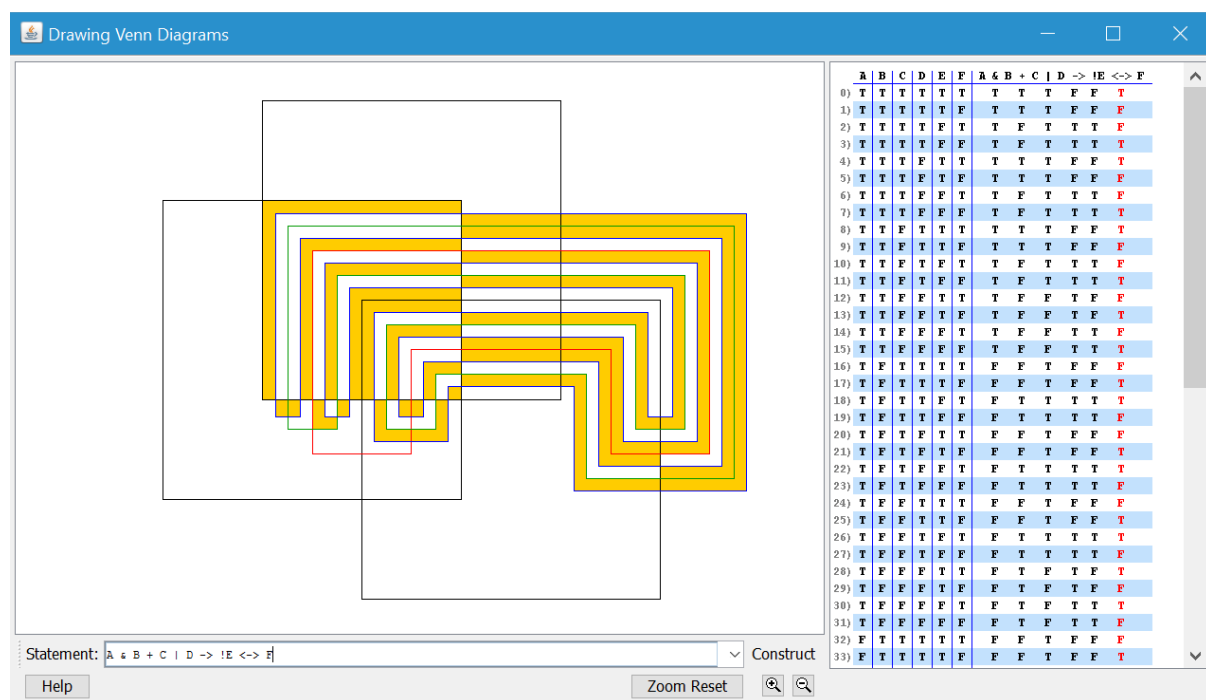


Figure 3.1: End result of the project.

From figure 3.1, you can see the “Statement “box. Here the user enters a propositional formula where input can be 1, 0 which represent universal and empty sets respectively, letters and symbols of Boolean algebra. The letters represent the sets in the diagram. As was discussed before, this application uses Venn’s construction, so the sets here are shown as squares and curves. The first letter represents the square on the left, second letter square on the top and third letter square on the bottom. Each consecutive new letter is represented by a new curve where each new curve has a different color from the previous curve. In this example, curves are added in a red, green and blue colors. After the propositional formula is typed, the user can press “Construct “button or simply press the “Enter “button on the keyboard. This will show a truth

table and a Venn diagram for corresponding formula. Figure 3.1 shows an example when " $A \& B + C \mid D \rightarrow !E \leftrightarrow F$ " is entered as a propositional formula. The truth values of the truth table are highlighted in the red color, and if truth value is "T" (true), then the corresponding region of that row is shaded in the Venn diagram. Any kind of wrong input is handled in this project as well. There are 3 possible cases for wrong input:

- If the user enters proposition and forgets to write a symbol between the letters, the program will give an error saying: "Missing connective at position n ".
- If the user enters any kind of character or symbol that is not recognized by the program, then it will give an error saying: "Illegal symbol at position n ".
- If the user enters a formula and forgets to add a letter between the symbols, the program will give an error saying: "Missing statement at position n ".

In all cases " n " is a number saying the actual position of mistake of the entered string. After the error message shown, the position at " n " will be highlighted. In figure 3.2, the user has entered " $A \ B \ \& \ C$ " as a statement. Here user forgot to add a symbol between the letters A and B. So an error message saying "Missing connective at position 2" is shown and position at 2 where user forgot to add symbol is highlighted.



Figure 3.2: "Missing connective" error.

The program has a feature of viewing all statements that were entered during its usage. At the right side of the "Statement box" there is a JComboBox that shows all previous inputs. Figure 3.3 shows an example of input history used in the program.

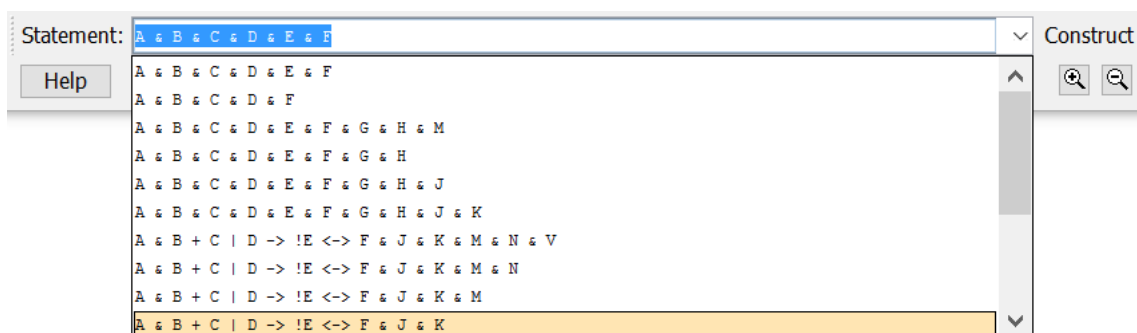


Figure 3.3: JComboBox of the input history.

In order to fully understand how this program works, quick usage guide is added which can be accessed anytime by pressing "Help" button on the bottom left part of the GUI. Figure 3.4 shows a quick usage guide after a "Help" button is pressed. It shows all possible inputs that are recognized by the program. The program takes letters to show propositions and recognizes all possible Boolean operations. A user can also use the mouse to click on the Venn diagram and

move it around. Venn diagram can also be moved by the help of JScrollBar located on the right and the bottom side of the Venn diagram. The user can also use JScrollBar to see the full truth table when it becomes really big. There is another feature for zooming in and zooming out the Venn diagram. This feature is discussed later in this chapter.

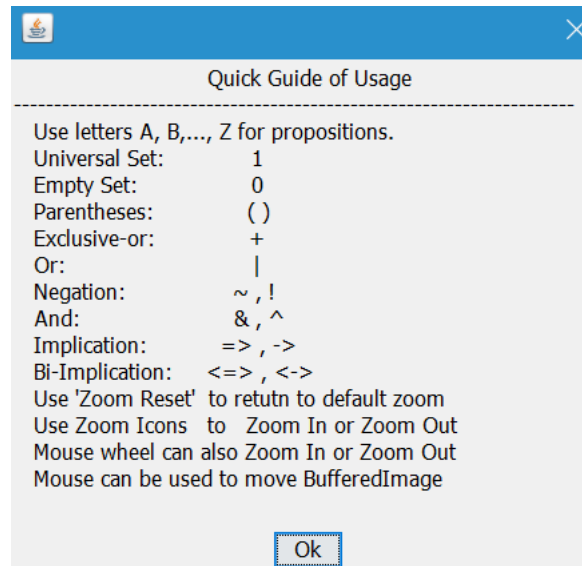


Figure 3.4: Quick guide of usage.

After the main features of the program were discussed to understand the project, the report starts to explain in details the design and implementation of this project.

3.2 Design

Implementing the Venn diagrams requires solving various problems. This project has 3 packages, 2 of these packages are responsible for analyzing the input and generating the truth table. The third package is responsible for drawing the Venn diagram, mapping the truth table with the diagram, filling the regions, zooming and creating a Graphical User Interface (GUI).

3.2.1 Truth table package

Creating a truth table consists of 2 packages. The first package is called “logic” and the second package is called “t t c” which stands for truth table constructor.

The logic package consists of several classes that are responsible for parsing the input. Parsing means to analyze the input with different characters and symbols, based on the provided rules. For example, if we want to compute simple mathematical equation $(3 + 4) * 5$ we need to follow basic rules of math to compute it. Firstly we need to compute $3 + 4$ within the bracket then we have to multiply it by 5. The same applies to logical functions. There is a

precedence of operations in which order the function has to be computed. Figure 3.5 shows the precedence of logical operators.

Precedence of Logical Operations	
Operator	Precedence
()	7
$\neg$	6
$\wedge$	5
XOR	4
$\vee$	3
$\rightarrow$	2
$\leftrightarrow$	1

Figure 3.5: Precedence of logical operations.

The package of truth table constructor also consists of several classes. This package is responsible for generating the truth table from the parsed input and to visualize it in the User Interface.

For this part of the project a source code was used and modified ^[5]. Many features from this source code were disabled as they were unnecessary for drawing Venn diagrams. Some of these features were, saving the truth table as an image, showing truth table graphically and in text format, changing the text font and showing truth values as letters or numbers (1/0). This features are not deleted but commented in case if someone would like to use them in the future.

3.2.2 Venn diagram package

Most of the work is done in this package. It contains the `MainInterface`, `Mapper`, `VennsConstruction`, `FloodFill` and `BottomPanel` classes.

- `MainInterface` – as it seen from the name this is the main graphical user interface of the project. This class takes the input of logical formula and sends it to the logic package after which a truth table is generated. This class is also responsible for combining truth table, Venn diagram and Bottom Panel. It divides the GUI in proportions so that truth table, Venn diagram and bottom panel are perfectly fitted in the interface.
- `VennsConstruction` - this class has various responsibilities. The main task of this class is to draw Venn diagram for any n , using Venn's construction. Secondly, it stores locations in the format of points that are required for filling the regions. Also, this class has methods that are required for zooming.
- `Mapper` – this is the class where a mapping algorithm is written. It is responsible to map the regions of the Venn diagram with the lines of the truth table. It creates a new list of points where each point represents the location of a unique region of the diagram. After the point of the regions and lines of truth table are mapped, points are sent to the Flood fill algorithm to fill the regions of the Venn diagram.

- **FloodFill** – this class is responsible for filling the regions of the Venn diagram. A Flood fill algorithm was implemented in this class. It takes points from the Mapper class in the format of x and y coordinates, and fills the region in which these coordinate are located. After it has filled the regions, the image of the Venn diagram is updated and shown in the Main Interface.
- **BottomPanel** – this class is separate from the Main Interface mainly because it uses a different layout. The design of this layout could be modified with the help of Window Builder provided by Eclipse. The main objective of this class is to design and store the buttons.

In order to better understand the design of the application and interaction between the classes consider the system class diagram of the project shown in figure 3.6.

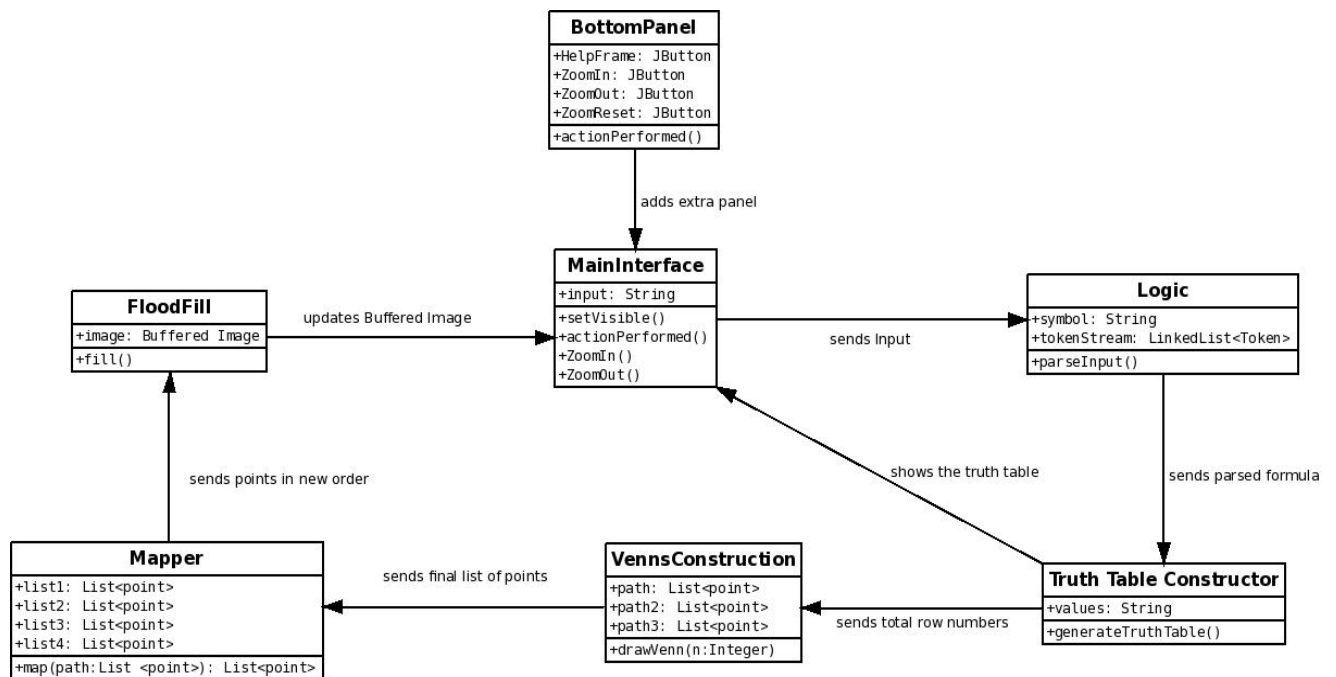


Figure 3.6: System class diagram of the program.

3.3 Implementation

After explaining the features and general design of the project, this section explains the whole implementation part of the Venn Diagrams.

This project was implemented in Java. And for developing the project Eclipse was used. It is an IDE (integrated development environment) which is mainly used by developers to create Java applications. Another advantage of Eclipse is its Window Builder Plugin. This plugin is integrated with Java swing that helped me to create and work on the design of graphical user interface. The implementation of this project could be divided into five parts. These are, *generating the truth table*, *drawing n-curved Venn diagram*, *mapping*, *filling* and *zooming*.

3.3.1 Generating the Truth table

To generate the truth table program has two packages “logic” and “ttc”, which are responsible for parsing the input and creating the truth table respectively. The “logic” package uses Shunting Yard algorithm to parse the input. This is a stack-based algorithm that parses the input based on infix notation. Infix notations in this project are logical operators that are written between the letters. The algorithm parses the input in two stages.

Firstly we declare and define the logical operators with their precedence based on rules of the Boolean algebra. Figure 3.7 shows the declared logical operators with their precedence that were implemented in the project.

Precedence of Logical Operators		
Name	Symbol (s)	Precedence
Negation	$\sim, !$	6
And	$\&, \wedge$	5
XOR	$+$	4
Or	$ $	3
Implication	$\Rightarrow, \rightarrow$	2
Bi-implication	$\Leftrightarrow, \leftrightarrow$	1

Figure 3.7: Precedence of logical operators implemented in the project.

Secondly, we parse the input using Shunting Yard algorithm. This algorithm reads the input and adds letters to the output queue and symbols to the stack based on their precedence ^[10]. When all symbols are added to the stack, it will pop up the symbols to the output queue and compute the result. Figure 3.8 shows the pseudo code of Shunting yard algorithm implemented in the project.

- For all the input tokens :
 - Read the next token;
 - If token is an operator (**x**) :
 - While there is an operator (**y**) at the top of the operators stack and either (**x**) is precedence is less or equal to (**y**), or (**x**) is precedence is less than (**y**) :
 - Pop (**y**) from the stack;
 - Add (**y**) output buffer;
 - Push (**x**) on the stack;
 - Else If token is left parenthesis, then push it on the stack;
 - Else If token is a right parenthesis :
 - Until the top token (from the stack) is left parenthesis, pop from the stack to the output buffer;
 - Also pop the left parenthesis but don't include it in the output buffer;
 - Else add token to output buffer.
- While there are still operator tokens in the stack, pop them to output

Figure 3.8: Pseudo code of Shunting Yard algorithm ^[10].

Let us consider that we have entered “A & B | C” as an input. The algorithm will act as follow:

- 1) Add A to the output queue
- 2) Add “&” to the stack
- 3) Add B to the output queue
- 4) Compare precedence of “|” with the last symbol of stack
- 5) Precedence of “|” is lower than “&” so add “&” to output queue and “|” to the stack
- 6) Add C to the output queue.
- 7) Add stack to the output queue.
- 8) Result: AB&C|

When the output queue is created, the program reads the output from left to right. It adds the letters to the stack, and when the program sees the operator, it computes the result of last two letters by using the last operator and replaces that two letters with their result. It continues until the output queue is finished. Let us consider how the previous example is computed after it was parsed.

- 1) The output queue is “ AB&C| “
- 2) Add A to the stack
- 3) Add B to the stack
- 4) Computer A & B and replace them with the result. Let us say, A & B = X
- 5) Add C to the stack
- 6) Compute X | C and replace it with the result.

After the input is parsed and ready to be correctly computed we need to replace the letters with the Boolean values *true* and *false* for 2^n possibilities. To do so, firstly we need to create a truth table for a corresponding formula with 2^n rows. Here n equals to a number of different letters. For each row, we have to replace the corresponding letter with the corresponding truth value of that row and compute it using the output queue. Computed result will be the truth value of the corresponding row. Let us consider the same logical formula “A &B |C”. Truth table with 2^3 rows is created as shown in figure 3.9. Every row has a column showing truth value of the letter. In the third row, the program replaces A with true B with false and C with false. Then it computes the result of “A & B | C” as “true & false | false”. Results are computed for every row as shown in figure 3.9.

	A	B	C	A & B	C
0)	T	T	T	T	T
1)	T	T	F	T	F
2)	T	F	T	F	T
3)	T	F	F	F	F
4)	F	T	T	F	T
5)	F	T	F	F	F
6)	F	F	T	F	T
7)	F	F	F	F	F

Figure 3.9: Truth table for A & B | C.

For generating a truth table, a source was used with a very appealing graphical interface [5]. It was modified as it was discussed in the design section.

3.3.2 Drawing n-curved Venn diagram

After the truth table is generated, the program uses the total number of truth table rows to identify the number of sets. The number of sets (" n ") in the diagram equals to $\log_2(\text{number of truth table rows})$. Then, the algorithm in the VennConstruction class draws a Venn diagram for " n ". The Venn diagram is drawn on the buffered image provided by Java.

The algorithm starts by initializing an Integer value called displacement. This is a crucial value as it is used for drawing all parts of the diagram. Displacement equals to the width of the buffered image divided by 8. We divide the width by 8 to get the optimal size of the diagram in the GUI; its dependability from the width is explained in the zooming section. After displacement is initialized, the algorithm goes through two major steps. These are drawing the Venn diagram for 3 squares when $n \leq 3$ and adding curves to the squares when $n > 3$

Drawing the squares:

For drawing squares `g.drawRect(x, y, width, height)` function is used. Here x, y are coordinates of the squares. The width and height of the squares are equal to $3 * \text{displacement}$ and initial coordinates of the first square are $(50 + \text{displacement}, 40 + \text{displacement})$. Depending on the value of " n " there are 3 cases for adding squares to the diagram.

1. If $n \geq 1$, there will be at least 1 square. A square is drawn by setting initial coordinates.
2. If $n \geq 2$, there will be at least 2 squares. Since one square is drawn, the 2nd square will be drawn with the new coordinates as shown in figure 3.10.
3. If $n \geq 3$, similarly as the first 2 squares the 3rd square is drawn with the new coordinates as shown in figure 3.10.

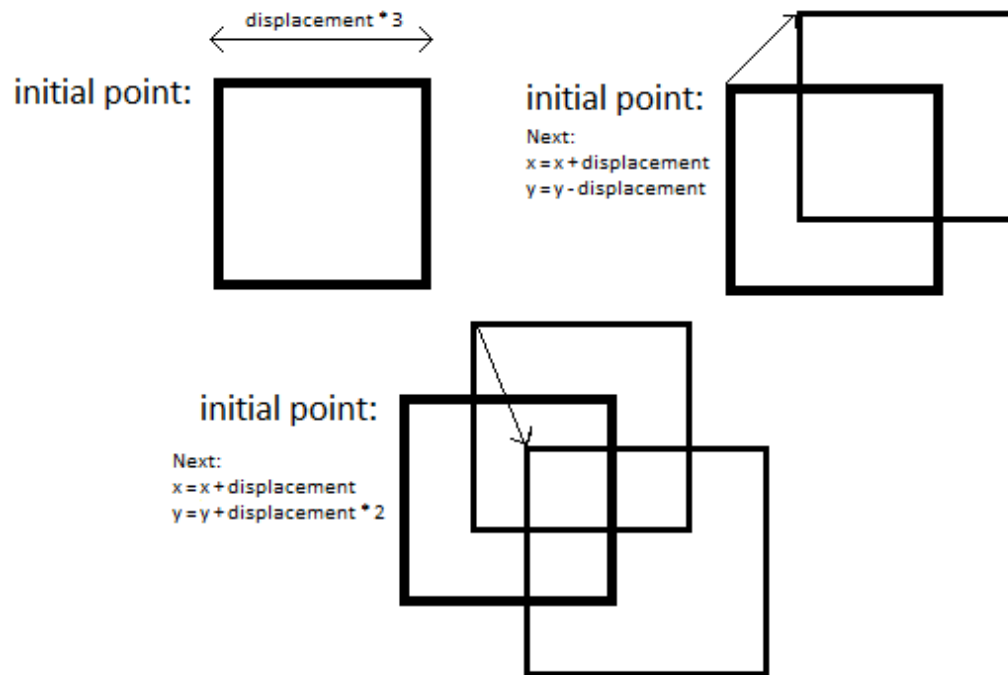


Figure 3.10: Drawing 3 squares.

Adding the curves:

When the third square is drawn, we get 4 points from it as shown in figure 3.11. Following these 4 points, the path is determined that can be followed to draw the curves. The program starts by drawing 2 lines from the first point to the second point and continues to draw 2 lines for the each next point as shown in figure 3.11. Along with the path, we save new points. When drawing 2 lines is finished, we add 2 additional lines above the starting point and ending points (1 and 4) that close the curve.

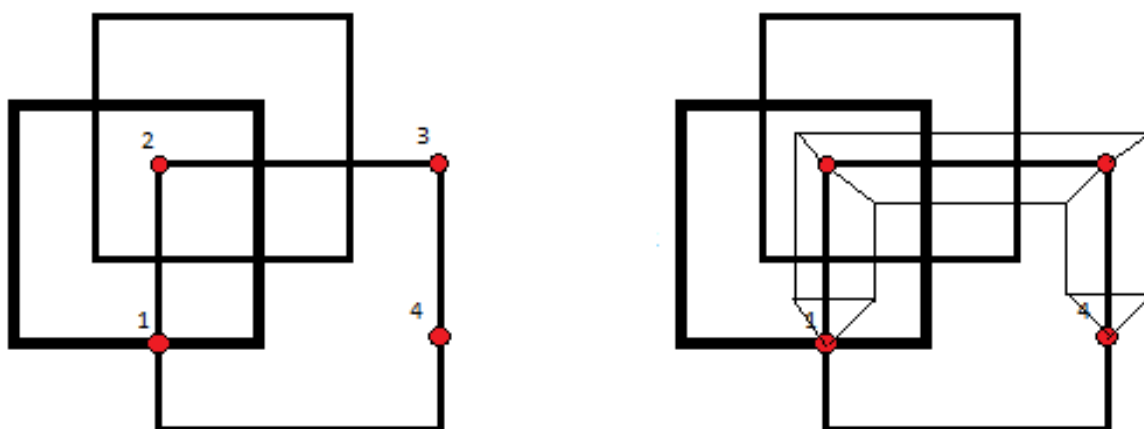


Figure 3.11: Adding the first curve from initial 4 points.

When drawing the curve is finalized, points are updated as shown in figure 3.12. The new points become the edges of the new curve. If the next curve needs to be added, we use the new points and follow the same rule of drawing 2 lines from point to next point and adding starting, ending lines. By this approach, we can draw any number of curves.

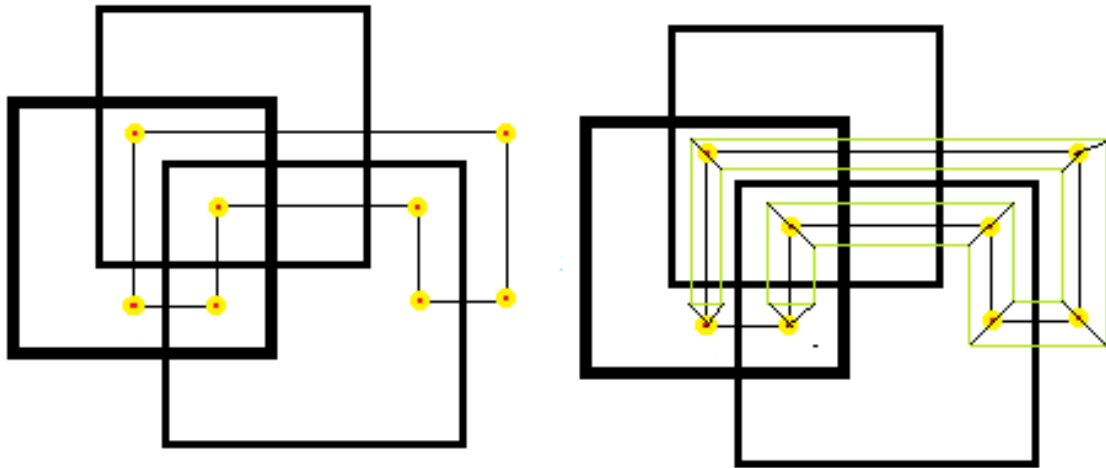


Figure 3.12: Updating the points and adding a new curve.

After 3 squares are drawn, the algorithm takes 4 points from the last square and adds them to the path which is in the format of `List<points>`. This is an `ArrayList` of points that represents location in (x,y) coordinate space. The algorithm starts by drawing starting and ending lines above the first and last points derived from the last square. Then it starts to draw 2 lines from one point to next point, but firstly, it has to identify in which direction they will go. To identify the direction of the line, the program compares x and y coordinates of 2 consecutive points taken from the path. Then it checks what was the last direction and what is the next direction. Because for each case the lines are drawn differently and coordinates of the new points are also different. So, to add the curves to the diagram, lines are drawn in four directions, and each direction has its own cases that have to be followed.

- Up direction ($x_1 = x_2$ & $y_1 > y_2$)

There are 5 possible cases when drawing lines in the up direction.

- 1) Starting case. It is a case when first 2 lines are drawn from 1st to 2nd point
- 2) Last direction: Left, Next direction: Right
- 3) Last direction: Left, Next direction: Left
- 4) Last direction: Right, Next direction Left
- 5) Last direction: Right, Next direction Right

- Down direction ($x_1 = x_2$ & $y_1 < y_2$)

Drawing lines in the down direction has same cases as drawing lines in the up direction with one difference. Instead of starting case, it has an ending case when 2 lines are drawn between the last 2 points.

- Left direction ($x_1 > x_2$ & $y_1 = y_2$)

There are 4 possible cases when drawing lines in the left direction.

- 1) Last direction: Up, Next direction: Down
- 2) Last direction: Up, Next direction: Up
- 3) Last direction: Down, Next direction: Up
- 4) Last direction: Down, Next direction: Down

- Right direction ($x_1 < x_2$ & $y_1 = y_2$)

Drawing lines in the right direction has same cases as drawing lines in the left direction.

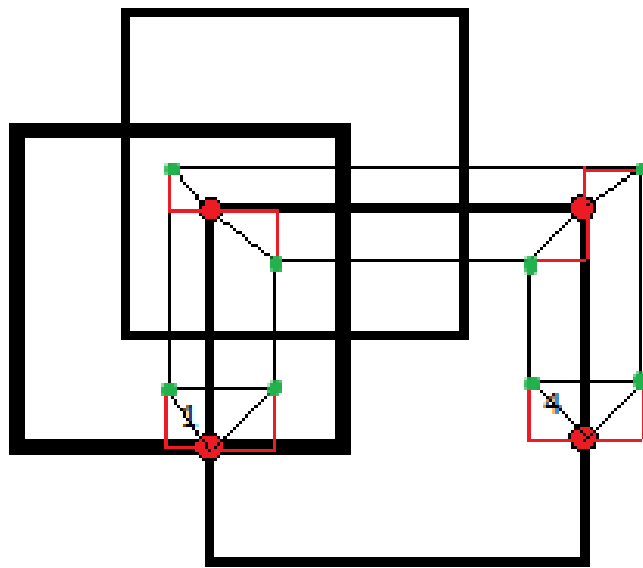


Figure 3.13: Computing new points from old points.

For every case, the algorithm computes new points by adding or subtracting displacement value from the old point. Consider figure 3.13, where the points in red are the old points. And to find the new points which are shown in green color, the algorithm has to identify the direction in which the lines will be drawn and a case that was discussed above. Because for every case, the algorithm has to add/subtract displacement value from x and y coordinates differently to find a new point. In figure 3.13 lines in red are representing displacement value used to find a new point (shown in green) from the old point (shown in red).

So for this example where $n = 4$, the algorithm starts by drawing starting and ending lines above the first and last points derived from the path. At this moment path stores 4 points derived from the third square as shown in figure 3.13. It draws these lines by using displacement value as shown in figure 3.14.


```

// draw starting and ending lines
if (i == 0) {
    g2d.drawLine(x1 - displacement, y1 - displacement, x1 + displacement,
        y1 - displacement);
} else if (i == path.size()) {
    g2d.drawLine(x2 - displacement, y2 - displacement, x2 + displacement,
        y2 - displacement);
}

```

Figure 3.14: Code for drawing starting and ending lines.

Then the algorithm starts to the process of drawing 2 lines from one point to the next point. Firstly, the algorithm identifies that it has to draw 2 lines in up direction then it identifies that it is our start case. So it computes new points for this case, and draws lines between these points. It adds the new points of one line to ArrayList called path2 and points of the second line to path3 as shown in figure 3.15.

```

// drawing two lines in up direction for start case
g2d.drawLine(x1 - displacement, y1 - displacement, x2 - displacement,
    y2 - displacement);
g2d.drawLine(x1 + displacement, y1 - displacement, x2 + displacement,
    y2 + displacement);

path2.add(new Point(x1 - displacement, y1 - displacement));
path3.add(new Point(x1 + displacement, y1 - displacement));

```

Figure 3.15: Code for drawing two lines between the points.

The algorithm continues to draw and update the points depending on each specific case. Every time new coordinates of one line are added to path2 and coordinates of the second line to path3. After the curves are drawn, points in the path are deleted. The algorithm then combines path2 and path3 and copies to the path. By this, the points are updated, and new points are now edges of the last curve. If the next curve needs to be added, the algorithm uses updated points stored in the path and follows the same rule of drawing the lines and updating the points. After each phase, displacement value is divided by 2, as each time the curves are getting 2 times closer to each other. Then the algorithm sets a new line color. By this, each new curve is in a different color. The colors in the algorithm are used in order of red, green, blue and purple.

3.3.3 Mapping

When drawing a Venn diagram is completed, the program needs to map the regions of Venn diagram to the rows of the truth table. Mapping also has 2 cases. These are mapping for 3 squares when $n \leq 3$ and mapping after the curves are added when $n > 3$.

1) Mapping for 3 squares ($n \leq 3$)

For this part, the coordinates of the Venn diagram regions were hardcoded. Figure 3.16 shows Venn diagrams when $n = 1, 2$ and 3. Here regions of the diagrams are shown in

numbers that represent rows of the truth table. By looking at this figure, we can see how regions of the Venn diagram are linked with the rows of the truth table. For all 3 values of "n", the coordinates are hardcoded and stored in a `List<points>`. For $n = 1$, two points are stored in a `List`. The first point stores the coordinates for the regions 0, and the second point coordinates for region 1 as shown in figure 3.16. If a truth table is generated for $n=1$, the algorithm finds which row of the truth table is true and gets coordinates from a `List<points>` for the corresponding row. For example, if row 0 in truth table is true, the algorithm gets the point from `List` at position 0 as well. So the points are hardcoded in the order which let us match them with rows of the truth table. The algorithm works in the same manner for $n=2$ and $n=3$. After the algorithm gets the points, it uses their coordinates to fill the associated regions.

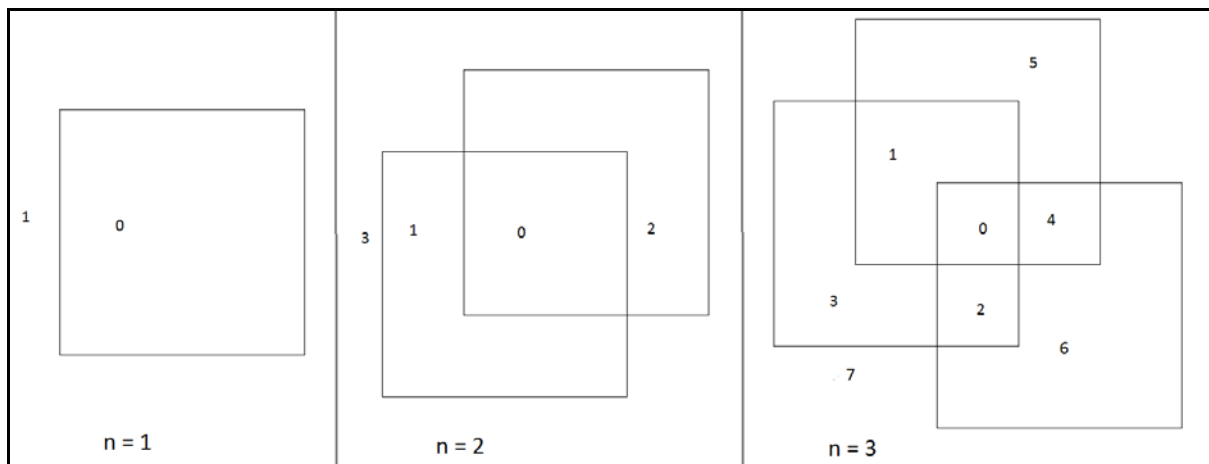


Figure 3.16: Relation of the Venn diagram regions with the truth table rows for $n=1$, 2, and 3.

2) Mapping after the curves are added ($n > 3$)

As it was mentioned before, every line of the truth table has associated region in the Venn diagram. However, for every new value of "n", the same region has a new corresponding row in a truth table. So it was required to find an algorithm that could constantly match the truth table rows with the regions of the Venn diagram for any number of "n".

After analyzing the connection between the truth tables and Venn diagrams, various relations were found. First of all, it is found that each quarter of the truth table rows have their associated regions in one of the four parts of the Venn diagram. Figure 3.17 shows four parts of the Venn diagram that are associated with that quarters. For example, if we have a truth table with 32 rows. First 8 rows of the truth table have their associated regions in the sector of Venn diagram shown with the line 1/4. Second 8 rows in 1/2, third in 3/4 and fourth in 4/4 sector.

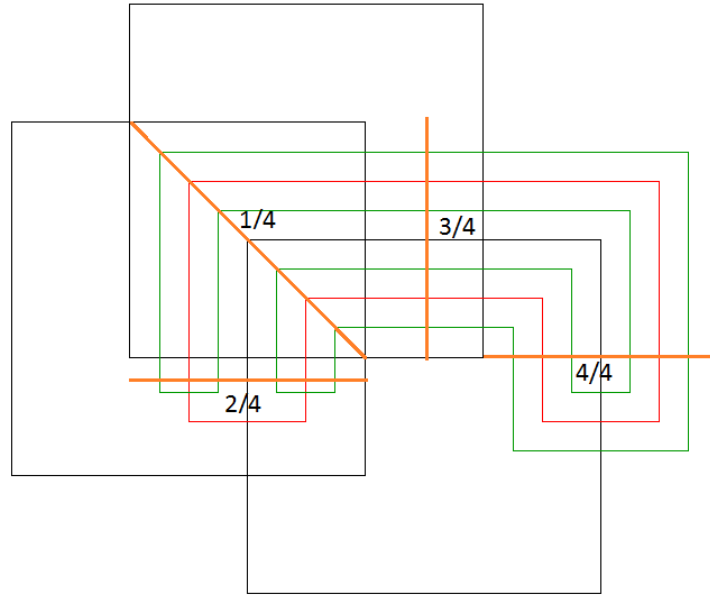


Figure 3.17: Sectors of the Venn diagram.

Figure 3.18 shows the regions of the Venn diagrams in numbers when $n=4$ and $n=5$. These numbers show to which row of the truth table that region corresponds. As it is seen from the figure the truth table rows are located quarterly in the 4 parts of the Venn diagram.

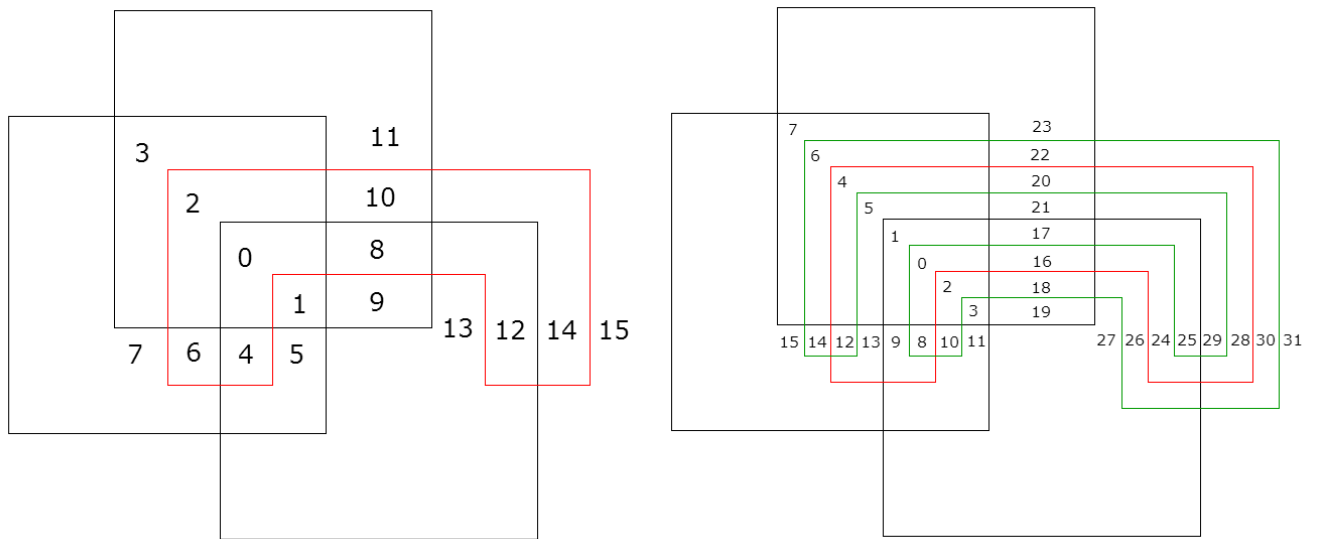


Figure 2.18: Venn diagram regions associated with the truth table rows.

In order to match these numbered regions with the rows of the truth table, the algorithm goes through several steps.

After the Venn diagram is drawn, the algorithm obtains the last set of points. As it was mentioned before, points are edges of the last curve. These points are stored in an order where the first point is the bottom left edge of the last curve, the second point is above it, and the next points are followed by tracing the line of the last curve. Figure 3.19 shows the order of last stored points when $n = 5$.

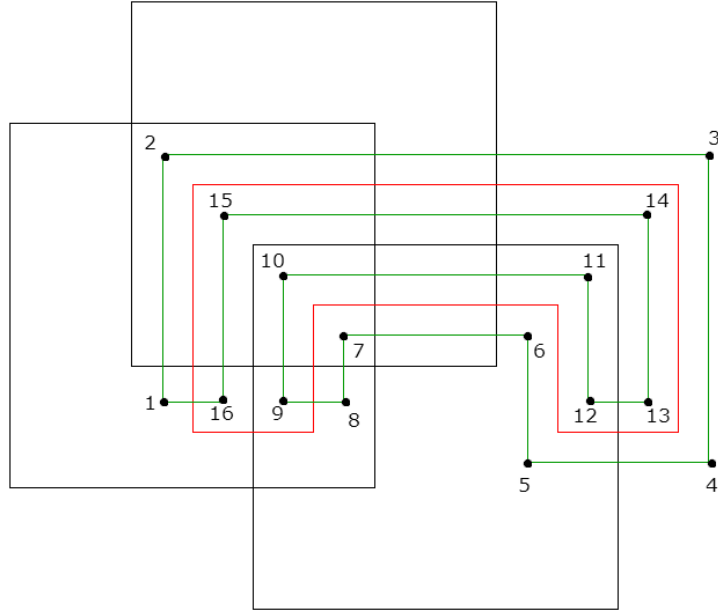


Figure 3.19: Order of the last stored points.

Now the algorithm uses the obtained points and creates a new pair of points that represent all regions of the Venn diagram. Then it stores these pairs of points in 4 lists. Where each list is of type `List<point>` representing one of four sectors of the Venn diagram.

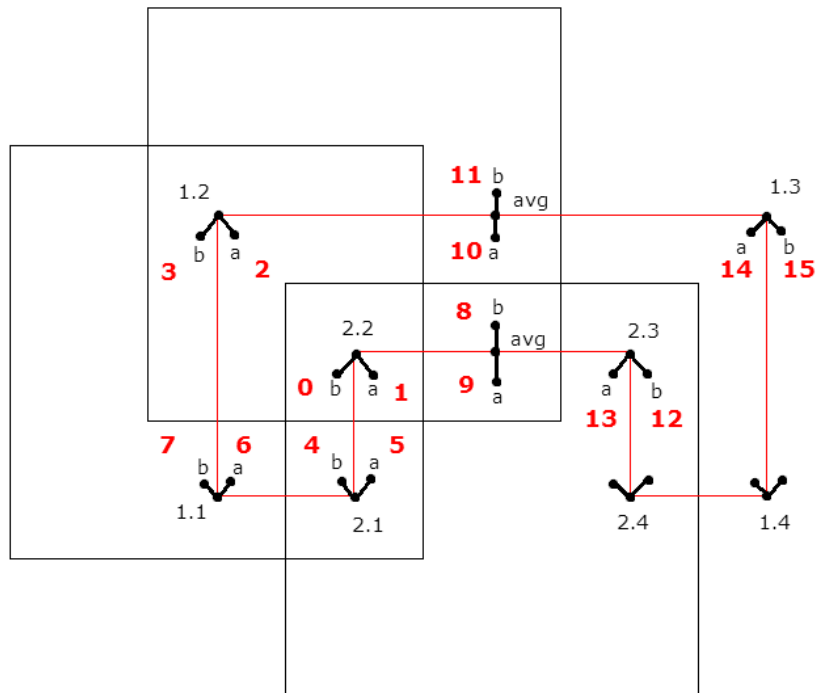


Figure 3.20: Pair points derived from the path.

Consider the figure 3.20. Each time the algorithm takes 4 points from the obtained list in a regular order then in reverse order. First four points are taken in a regular order shown as 1.1, 1.2, 1.3 and 1.4. Then the algorithm takes next 4 points, but this time, it uses them

in reverse order. Next 4 points are 5th 6th 7th and 8th points of the Venn diagram. So the algorithm firstly takes the 8th point and then the 7th 6th and 5th points. These points are shown as 2.4, 2.3, 2.2 and 2.1. Each time when algorithm takes 4 points, it finds an average point between 2nd and 3rd points. In figure 3.20, these points are shown as "avg". On each iteration, the algorithm creates pair points "a" and "b" for 1st, 2nd, 3rd and avg points as shown in figure 3.20. The pair points of the 1st points, 2nd points, 3rd points and avg points are added to a list2, list1, list4 and list3 respectively. When the algorithm takes 4 points in regular order, it adds a pair points to the list in order of a, b and when in reverse order it adds them in the order of b, a. In figure 3.20, regions are shown in numbers, and numbers show to which row of the truth table that region corresponds. So for the figure 3.20, we will get 4 lists with points that will represent the regions as follow:

List1={2, 3, 0, 1}, List2={6, 7, 4, 5}, List3={10, 11, 8, 9},
List4={14, 15, 12, 13}

Now, the algorithm has to arrange the points in the list in order that let us match them to the truth table rows.

For each four lists, the algorithm picks two points from the end and then adds two points from the beginning to the temporary list. It continues in this manner until all points are used. Then it deletes the old list and copies the temporary list to the old list. When 4 lists are updated, the algorithm combines the lists in the order of list1, list2, list3 and list4. For the previous example, lists will be as follow:

List1={0, 1, 2, 3}, List2={4, 5, 6, 7}, List3={8, 9, 10, 11},
List4={12, 13, 14, 15}

After combining 4 lists,

finalList={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}.

Now points are ordered in a list in a manner that let us match them to the truth table. For example, if a row in the truth table at position 7 is true, the algorithm gets the point from finalList at position 7 and uses its coordinates to fill the region.

3.3.4 Filling the regions

To fill the regions of the Venn diagram Flood fill algorithm was used. This is a stack-based recursive algorithm ^[9]. The algorithm takes a starting point and expands to the last part of the available image. When it reaches the border of the image, it comes back along with coloring each pixel. Figure 3.21 shows how Flood fill algorithm works in steps.

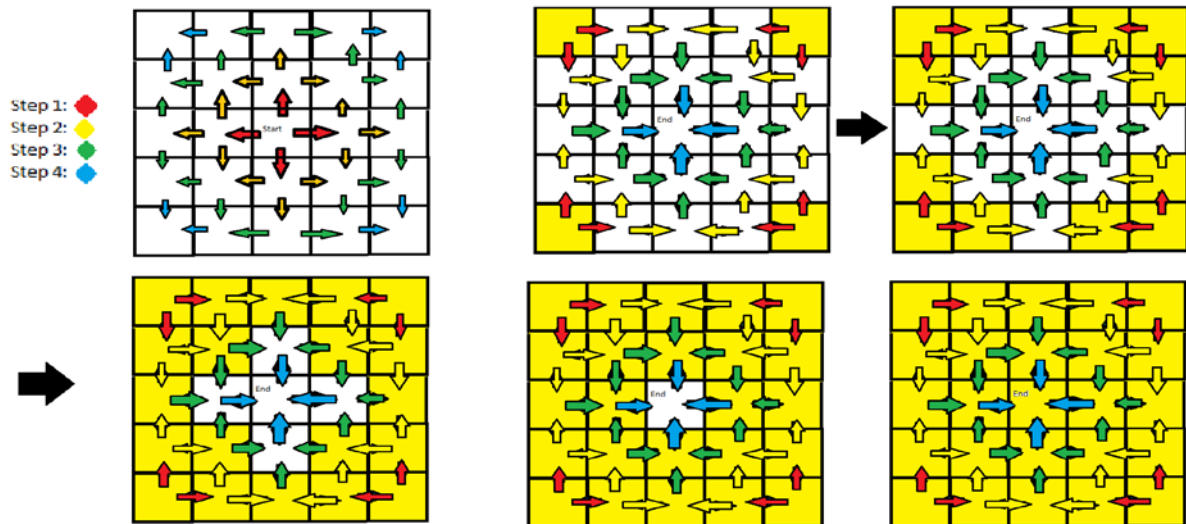


Figure 3.21: The logic of the Flood fill algorithm.

The Flood fill algorithm was implemented on the buffered image with a white background where the Venn diagram is drawn. Firstly we pass the start point coordinates to the Flood fill algorithm. For a given coordinate we call the function for its 4 neighbors as shown in figure 3.21. If any neighbor is colored other than white color, then we do not call the function on that. Otherwise, we call the function again. In this manner, we reach at the end pixels. From there we come back by coloring pixels, and hence, when all callbacks are done the image is updated in the GUI. To check whether the pixel is white, `color.getRGB()` and to change the color of the pixel `color.setRGB()` functions provided by Java were used. Figure 3.22 shows the pseudo code of the Flood fill algorithm that was implemented in the project.

```
Flood-fill (pixel, target-color, replacement-color):
  1. If target-color is equal to replacement-color, return.
  2. If the color of pixel is not equal to target-color, return.
  3. Set the color of pixel to replacement-color.
  4. Perform Flood-fill (one step to the south of pixel, target-color, replacement-color).
     Perform Flood-fill (one step to the north of pixel, target-color, replacement-color).
     Perform Flood-fill (one step to the west of pixel, target-color, replacement-color).
     Perform Flood-fill (one step to the east of pixel, target-color, replacement-color).
  5. Return.
```

Figure 3.22: Pseudo-code of the Flood fill algorithm [9].

As the Flood fill algorithm uses the stack memory, it was necessary to increase the stack size by using Java VM argument “-XSS “.

3.3.5 Zooming

When the user enters a formula with 9 or more propositional variables, the Venn diagram will have 6 or more curves, and there will be no pixels left to show the regions of the Venn diagram. To overcome this problem, it was decided to implement a zooming function that increases the size of the buffered image and redraws the Venn diagram according to its size. By using this approach, the size of the Venn diagram will be increased on the larger buffered image and eventually there will be more pixels between the curves.

Before the zooming function was implemented, the algorithm drew the Venn diagram that was independent of the buffered image's size. The program used a constant position and a constant size of the Venn diagram. After the zooming function was implemented, the program regularly used a value called `displacement`. This value is used to identify the position of the Venn diagram on the buffered image, to draw initial squares and curves of the diagram. As it was mentioned before, $\text{displacement} = \text{width} / 8$. So as the size of the buffered image increases, the `displacement` value also increases. In the section of drawing n-curved Venn diagram, it is shown how the program draws initial 3 squares and identifies their position by using `displacement` value. It is also explained how the program gets the new points by using `displacement` value to draw the curves of the Venn diagram. So, when the user zooms in, the program increases the width and height of the buffered image and redraws the Venn diagram on a new buffered image based on its size. In fact, the hardest part of implementing zooming was to re-code an algorithm that drew n-curved Venn diagram and make it dependable on the size of the buffered image. Another thing that needs to be changed was hardcoded points used to map truth table with the Venn diagram. As the size of the buffered image changed, hardcoded points could refer to different regions, so, these points were made dependable on the `displacement` value as well.

Two screenshots of the program are shown in figure 3.23 and 3.24. Screenshots are showing a Venn diagram with 9 propositional variables. The first picture shows how the Venn diagram looks before it was zoomed in and the second picture shows the result after it was zoomed in.

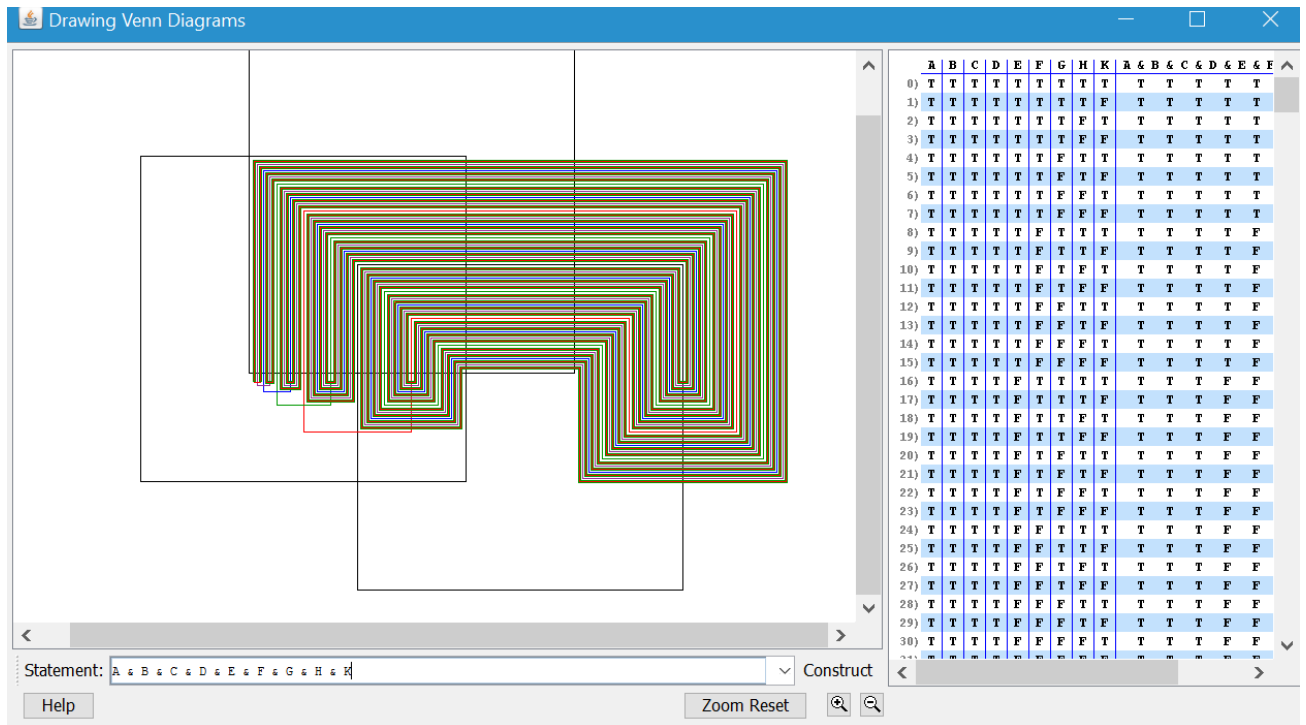


Figure 3.23: Venn diagram before zooming.

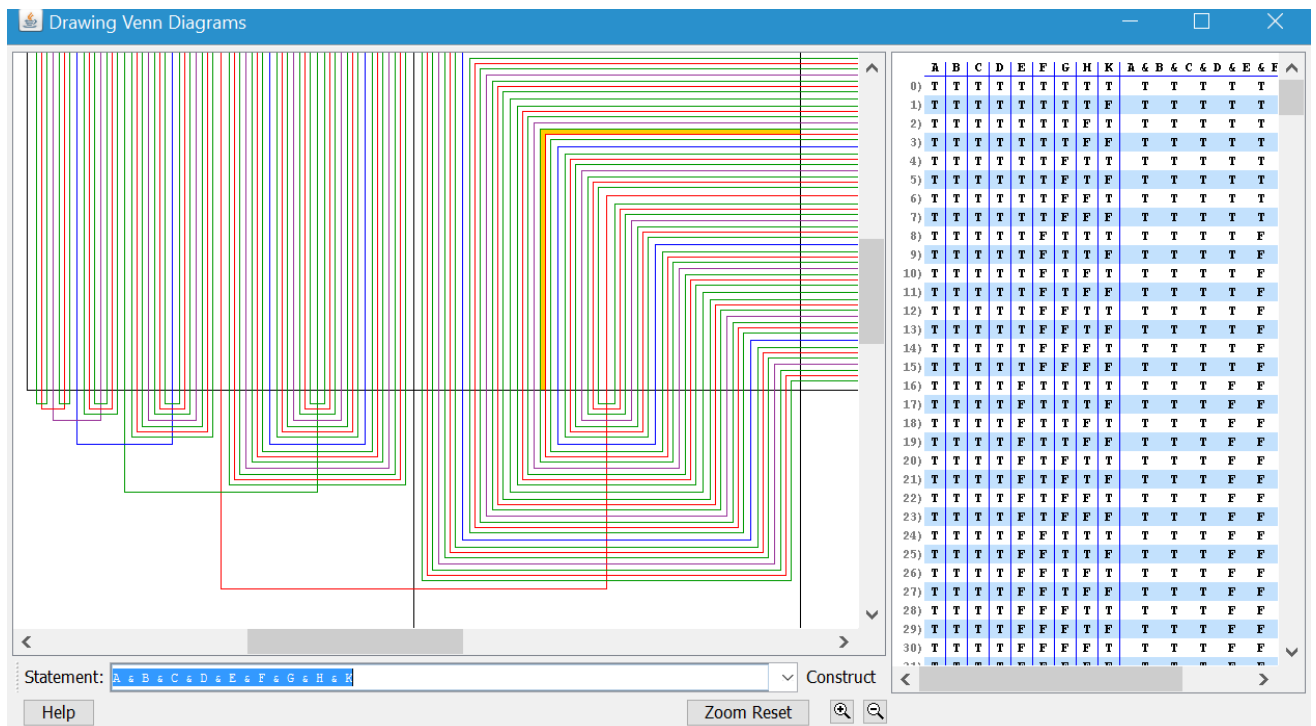


Figure 3.24: Venn diagram after zooming.

As the buffered image is getting bigger each time, default heap size allocated for the buffered image was increased to avoid any kind of error related to the heap size. To increase heap size Java VM argument “-Xmx” was used.

Chapter 4

Testing and Evaluation

This chapter covers test cases used to evaluate correctness of the program and results on how fast the program works for the large inputs.

4.1 Testing

In order to check that the program works successfully, various inputs tested to check whether the truth tables and the Venn diagrams are correctly generated.

Here are some examples of recently tested inputs:

1. $A \& B$
2. $C \& 1$
3. $D \& 1$
4. $X \mid 1$
5. $B \& A \mid C \mid D \& 0$
6. $D \& A \mid C + B$
7. $A \mid B \rightarrow C$
8. $D + C \leftrightarrow A$
9. $C \leftrightarrow !A \& (P \rightarrow (D \mid !Z))$
10. $K + (H \mid B \rightarrow V)$
11. $P \mid Q \& V \mid X + B + J \mid S \rightarrow F$
12. $X \leftrightarrow (N \rightarrow B \mid !G \& (S \mid N \mid X))$
13. $(A \& B \mid C \& ((D \rightarrow E \& G) + !G \& H \& !(J \& K + L \& M) + N \& O) \mid P) \rightarrow 1$
14. $A \leftrightarrow B \& 0$
15. $B \& C \& D \& E \& F \& G \& H \& J \& K \& L \& M \& N \& B \& V \& X \& Z \& A$
16. $N \mid B + (((!B \& !C \mid D \& F) \rightarrow X) \rightarrow M)$
17. $P \mid Q \& R \& S \& N + M \& X$
18. $W \& N \& V \mid A \rightarrow C \leftrightarrow F$
19. $1 \& !X$
20. $B \& !X \& S + C$

In fact, many more inputs were tested during the work on the project, and in all cases program works correctly. However, there was found a small bug after the submission. When a user inputs a large formula, and there are no pixels left between the curves, the program cannot fill some regions. This could be easily overcome by zooming the Venn diagram. After the user zooms in, there will be pixels between the curves and all regions will be correctly filled.

4.2 Evaluation

This section shows the performance results of the program. It demonstrates how much time it takes to generate the truth tables and the Venn diagrams separately. It also shows how much time it takes for the whole program to run. Results are shown in tables and graphs.

Up to 20 propositional variables were used to evaluate the performance of the program. And to check the run time of the program `System.nanoTime()` function provided by Java was used.

- Runtime results for generating the truth tables:

Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)
1	0.0001	6	0.0003	11	0.0018	16	0.0037
2	0.0001	7	0.0008	12	0.0020	17	0.0040
3	0.0002	8	0.0016	13	0.0026	18	0.0048
4	0.0002	9	0.0016	14	0.0029	19	0.0050
5	0.0002	10	0.0016	15	0.0030	20	0.0060

Figure 3.1: Tabulated runtime results for generating the truth tables.

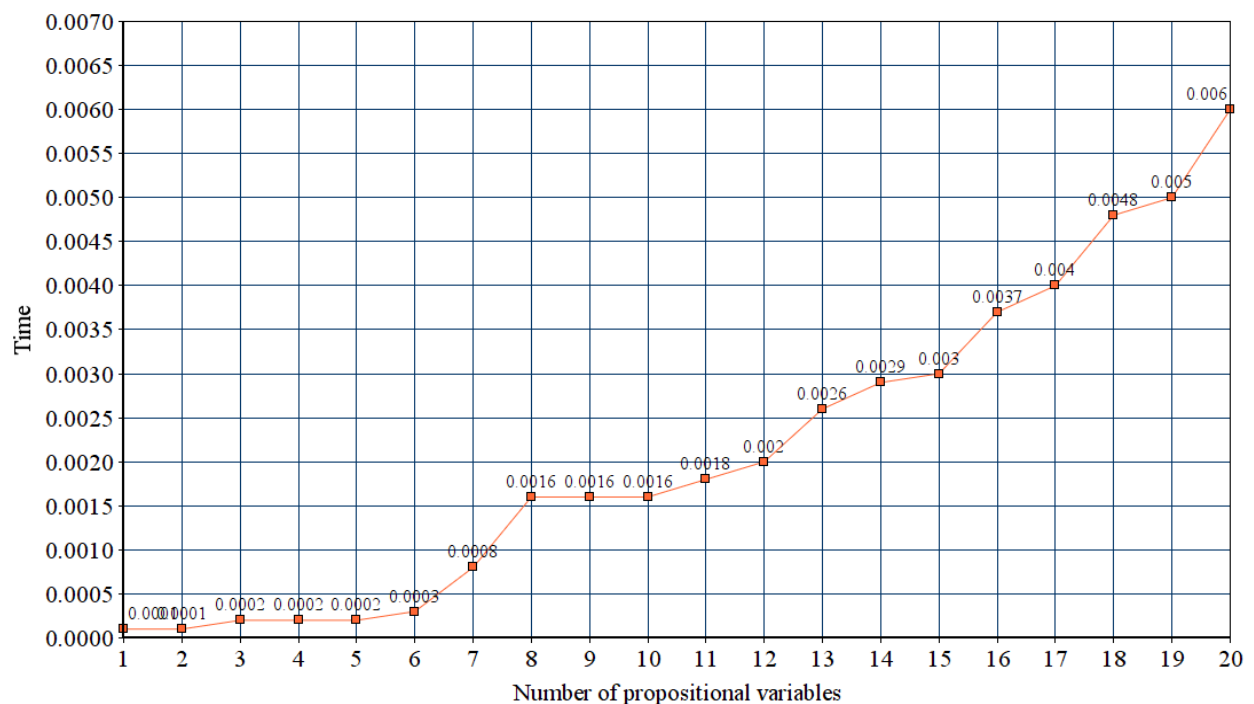


Figure 4.2: Graphical runtime results for generating the truth tables.

- Runtime results for drawing the Venn diagrams:

Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)
1	0.0023	6	0.0050	11	0.0067	16	0.1007
2	0.0025	7	0.0024	12	0.0122	17	0.2120
3	0.0016	8	0.0030	13	0.0188	18	0.3915
4	0.0019	9	0.0033	14	0.0325	19	0.6852
5	0.0030	10	0.0050	15	0.0648	20	1.3817

Figure 4.3: Tabulated runtime results for drawing the Venn diagrams.

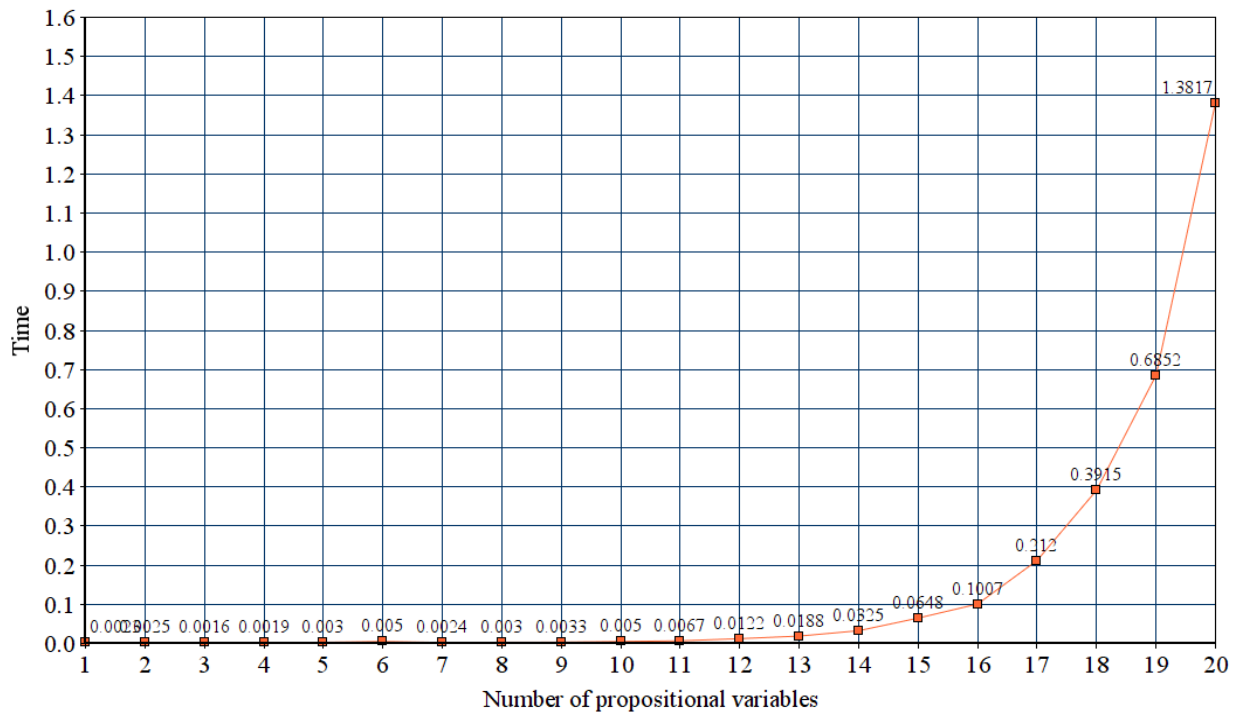


Figure 4.4: Graphical runtime results for drawing the Venn diagrams.

- The time required for the whole program to run:

Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)	Number of variables	Time (seconds)
1	0.0102	6	0.0146	11	0.0162	16	0.1114
2	0.0124	7	0.0132	12	0.0231	17	0.2456
3	0.0122	8	0.0140	13	0.0261	18	0.4120
4	0.0131	9	0.0155	14	0.0378	19	0.8613
5	0.0138	10	0.0159	15	0.0740	20	1.5600

Figure 4.5: Tabulated runtime results of the whole program.

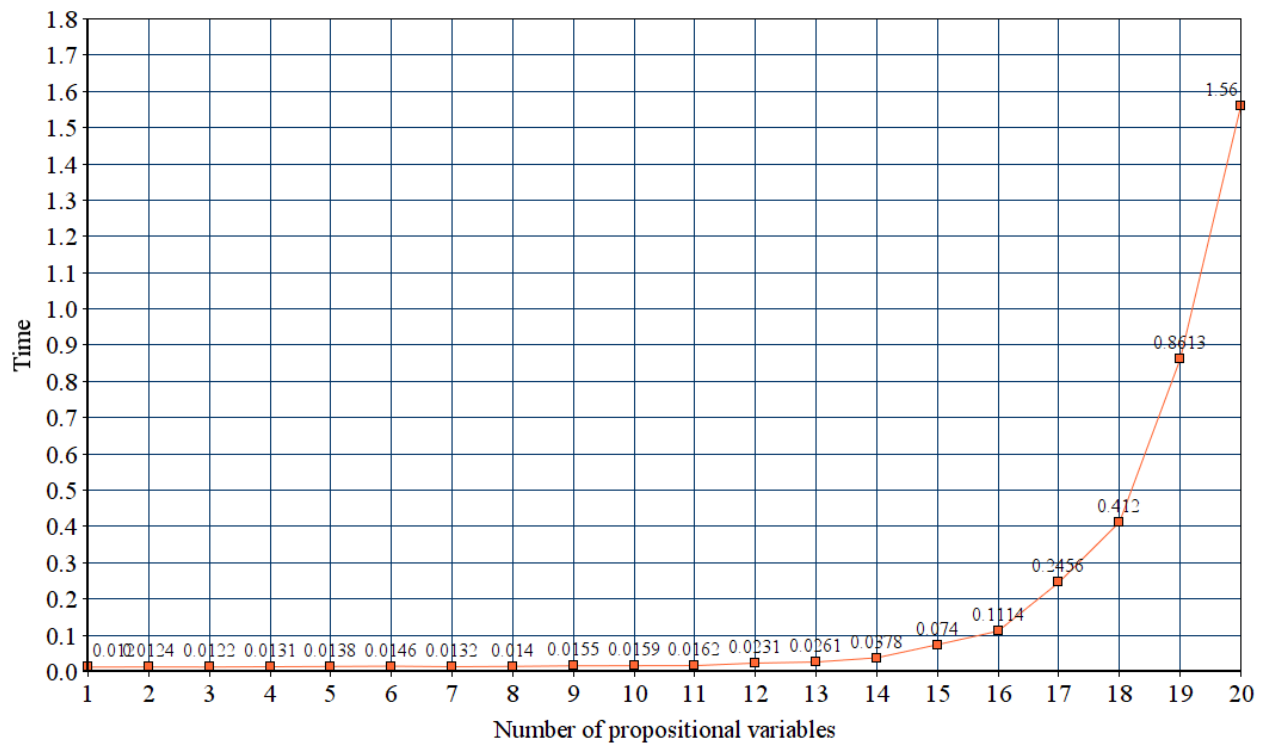


Figure 4.6: Graphical runtime results of the whole program.

As the rows of the truth table and regions of the Venn diagram are increasing exponentially, the run time of the program also increases exponentially. But even for a truth table with a million lines and the Venn diagram with a million regions (2^{20}), the program needs a second to generate them.

Chapter 5

Conclusion and Future Works

5.1 Conclusion

The report provides the background information about the Boolean algebra, Venn diagrams and the relation between them. It further demonstrates what the program does and the features it has. The report then carefully explains how the program was designed by showing the relations between the Java classes. Then it describes how the project was implemented by explaining the approaches and the algorithms that were used. Finally, the report provides the performance results of the program.

The program takes logical functions, parses and generates the truth tables for them. Then it draws corresponding Venn diagrams to represent all relations between the sets for the corresponding logical formula. Implemented algorithm can draw the infinite number of the Venn diagrams. So if the diagrams get very big, and there are not enough pixels to draw the Venn diagrams, a zooming function was implemented that allows to zoom and see all parts of it.

5.2 Future works

The most complicated parts of this project were successfully completed, with an extra feature of zooming. However, due to the time limitations, zooming was the last feature that was implemented. Additional work that could improve the project would be labeling that shows to which square or curve the propositional letter corresponds. Figure 5.1 shows how this feature might look like.

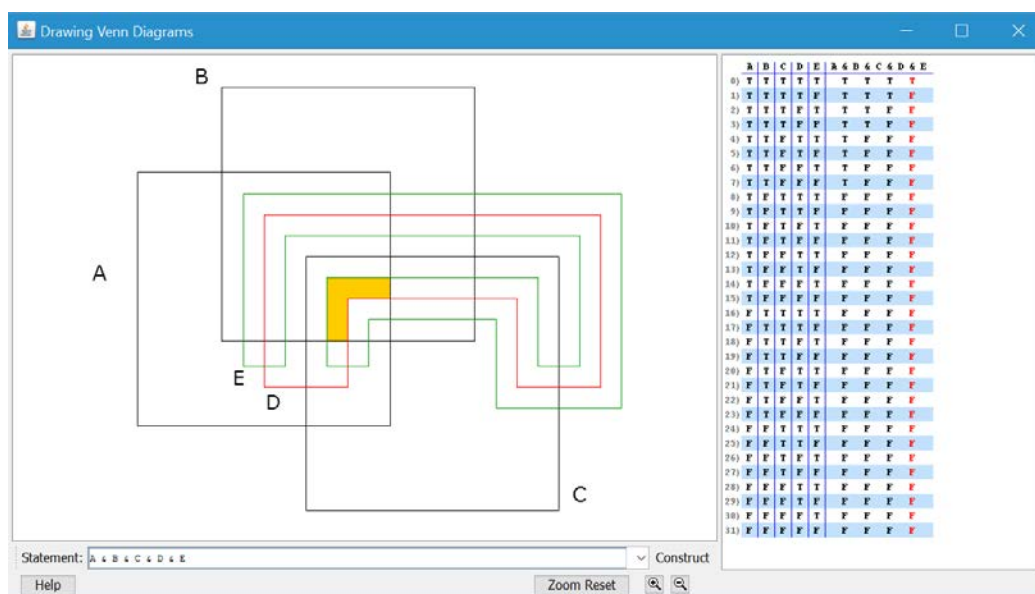


Figure 5.1: Labeling.

References

- [1] Blackwell, Alan F, 2001, *Thinking with diagrams*. Dordrecht : Kluwer Academic Publishers. [Accessed 12 April 2016]. Available at: <https://books.google.co.uk/books?id=MzuzBwAAQBAJ&pg=PT42&lpg=PT42&dq=diagrammatic+reasoning+keith&source=bl&ots=ACHixPP6oA&sig=yRm9G16QyxaVMk0r96YU3PdCqyc&hl=ru&sa=X&ved=0ahUKEwjK-PuzuZPMAhUHWSwKHQl7Dg8Q6AEIMzAD#v=onepage&q=diagrammatic%20reasoning%20keith&f=false>
- [2] Rusky, Frank and Wetson, Matt, 2005, A Survey of Venn Diagrams. *Combinatorics.org* [online]. 2005. [Accessed 8 April 2016]. Available at: <http://www.combinatorics.org/files/Surveys/ds5/VennWhatEJC.html>
- [3] D. W. Henderson, Venn diagrams for more than four classes, *American Mathematical Monthly*, 70 (1963), 424-2-6.
- [4] Enderton, H. (2001). *A Mathematical Introduction to Logic*, second edition, New York: Harcourt Academic Press. ISBN 0-12-238452-0
- [5] Borowski, Brian, 2011, Truth Table Constructor. *Brian-borowski.com* [online]. 2011. [Accessed 14 April 2016]. Available at: <http://www.brian-borowski.com/software/truth/>
- [6] Weisstein, Eric W. "Venn Diagram." From MathWorld--A Wolfram [online]. 2009. [Accessed 16 April 2016]. Available at: <http://mathworld.wolfram.com/VennDiagram.html>
- [7] Edwards, A.W.F. (2004). *Cogwheels of the mind: the story of Venn diagrams*. JHU Press. ISBN 978-0-8018-7434-5. [Accessed 17 April 2016]. Available at: http://books.google.com/books?id=7_0Thy4V3JIC
- [8] Aron, Jacob, 2012, Discover the beauty of extreme Venn diagrams. *New Scientist* [online]. 2012. [Accessed 16 April 2016]. Available at: <https://www.newscientist.com/gallery/venn/>

[9] Shaw, J. R. (2004). *QuickFill: An efficient flood fill algorithm*. [online]. 2004. [Accessed 17 April 2016]. Available at:
<http://www.codeproject.com/Articles/6017/QuickFill-An-efficient-flood-fill-algorithm>

[10] Theodore Norvell. *Parsing Expressions by Recursive Descent*. 1999–2001. [online]. 2001. [Accessed 19 April 2016]. Available at:
http://www.engr.mun.ca/~theo/Misc/exp_parsing.htm#shunting_yard